Squeezing Every FLOP: Single-GPU Training Efficiency Optimization for Vision-Language Models

A Case Study on SiQ-VL (0.5B) with NVIDIA Blackwell

Duo An
duo.an@example.com

May 2026

Abstract

We present a systematic study of single-GPU training efficiency optimization for SiQ-VL, a small-scale (0.5B parameter) vision-language model trained on an NVIDIA RTX PRO 6000 Blackwell GPU (96 GB VRAM). Through 15 iterations of measured optimizations spanning kernel fusion, data strategies, and batch scaling, we improve effective throughput from 21.7K to 107.4K real tokens/second — a **4.95× speedup**. We identify the throughput ceiling imposed by Model FLOPs Utilization (MFU ≈ 22.7%) for this model scale, demonstrate that length bucketing eliminates 95% of padding waste, and show that cuTile (TileGym) kernels provide a further 14–23% speedup at fixed tensor shapes. All measurements use real training data with precise attention-mask accounting, correcting earlier estimates based on synthetic benchmarks.

Keywords: Vision-Language Models, Training Efficiency, GPU Kernel Optimization, cuTile, Blackwell, Sequence Packing

Contents

1 Introduction

Training vision-language models (VLMs) efficiently on a single GPU is crucial for researchers with limited hardware budgets. While much attention has been paid to distributed training at scale, the *single-GPU efficiency frontier* determines the baseline throughput that distributed methods multiply.

This report documents the complete optimization journey of **SiQ-VL**, a compact VLM combining a SigLIP-2 vision encoder with a Qwen2.5 language model. We focus exclusively on engineering optimizations that preserve model quality (identical loss curves) while maximizing tokens processed per second.

1.1 Contributions

1. A **reproducible benchmark** (`benchmark_real_efficiency.py`) that measures Real tok/s, Pos/s, Pad%, and Loss% on actual training data.
2. Quantitative evidence that **length bucketing** at large batch sizes (B≥64) reduces padding waste to <5%, making sequence packing only marginally beneficial (+4.6%).
3. Demonstration that NVIDIA's **cuTile DSL** (via TileGym) achieves 23% kernel speedup but *requires* quantized tensor shapes to avoid catastrophic JIT overhead.
4. A complete **MFU analysis** confirming that 0.5B-scale models are memory-bandwidth bound at 19–23% MFU, with throughput saturating at B≥64.

2 System Setup

2.1 Hardware

Table 1: Hardware configuration

Component	Specification
GPU	NVIDIA RTX PRO 6000 Blackwell Server Edition
VRAM	96 GB GDDR7
SMs	188 @ 2430 MHz
Peak BF16 Tensor	936 TFLOP/s (dense)
Memory Bandwidth	1.79 TB/s
CUDA / Driver	CUDA 13.0, Driver 580.126

2.2 Model Architecture

Table 2: SiQ-VL model configuration

Component	Architecture	Parameters
Vision Encoder	SigLIP2-base-patch16-224	92.9M (15.8%)
Language Model	Qwen2.5-0.5B-Instruct	494.0M (83.8%)
Projector	2-layer MLP + Pixel Shuffle	2.8M (0.5%)
Total		**589.7M**

Training stages:

- **Stage 1** (Projector alignment): Vision encoder and LLM frozen; only the 2.8M projector trains. FLOPs/token = $4NP$ (forward + activation backward).
- **Stage 2** (LLM fine-tuning): Vision encoder frozen; LLM trains (full or LoRA). FLOPs/token = $6NP$ (full forward + backward).

2.3 Software Stack

- PyTorch 2.9.1 with `torch.compile` support
- HuggingFace Transformers 5.3.0
- TileGym (NVIDIA cuTile DSL kernels for Blackwell)
- Dataset: `lmms-lab/LLaVA-OneVision-Data / sharegpt4v(coco)` (50K samples)

3 Methodology

3.1 Metrics

We define four metrics measured *exactly* from tensor values:

$$\text{Real tok/s} = \frac{\sum_i \texttt{attention\_mask}_i\texttt{.sum()}}{\Delta t} \tag{1}$$

$$\text{Pos/s (hw)} = \frac{\sum_i B_i \times N_i}{\Delta t} \tag{2}$$

$$\text{Pad\%} = 1 - \frac{\text{Real tok/s}}{\text{Pos/s}} \tag{3}$$

$$\text{MFU} = \frac{\text{Real tok/s} \times F_{\text{tok}}}{\text{Peak TFLOP/s}} \tag{4}$$

where $F_{\text{tok}} = 4 \times P = 4 \times 494\text{M} = 1.976$ GFLOP for Stage 1.

3.2 Measurement Protocol

Each configuration is measured as follows:

1. Load real dataset (not synthetic) with actual images.
2. Run W warmup steps (2–10 depending on JIT overhead).
3. Run $S = 10$ measured steps, recording per-step: wall time (`cuda.synchronize` bracketed), `attention_mask.sum()`, `(labels != -100).sum()`, $B \times N$.
4. Report aggregated metrics over all S steps.

4 Optimization Techniques

4.1 Length Bucketing

The standard VLM training pipeline pads all sequences in a batch to the longest. With random sampling, padding waste scales with batch size (18–20% at B=32). **Length bucketing** (HuggingFace `group_by_length`) sorts samples by estimated token length, ensuring each batch contains similar-length sequences.

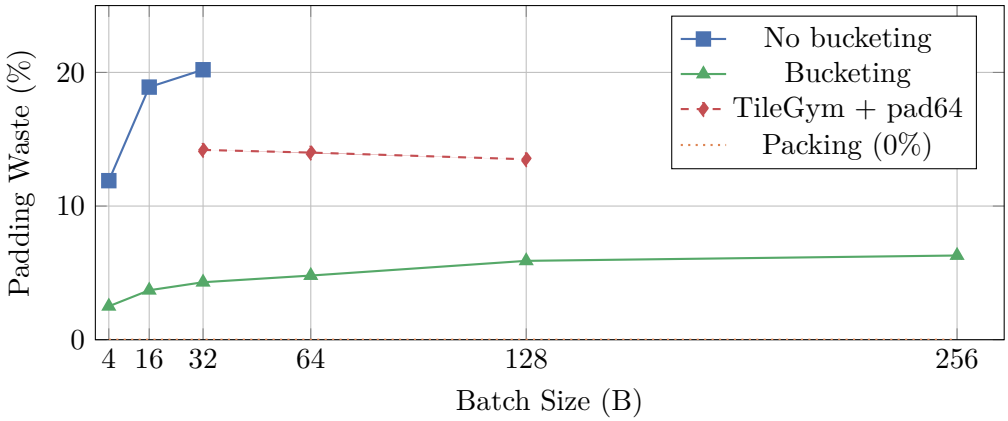

Figure 1: Padding waste vs. batch size. Bucketing reduces waste from 20% to <6%. TileGym's `pad_to_multiple_of=64` introduces 13–14% intentional padding as a trade-off for kernel efficiency. Packing eliminates padding entirely.

4.2 Sequence Packing

Packing concatenates multiple short sequences into fixed-length bins (N=1024), using position ID resets to maintain document boundaries. This eliminates padding at the cost of more complex attention masking.

4.3 TileGym (cuTile DSL Kernels)

NVIDIA's cuTile DSL generates Blackwell-optimized kernels for matmul, attention, and fused operations. We use TileGym to patch Qwen2's attention and MLP layers:

```
from tilegym.transformers import apply_tilegym_kernel_to_qwen2
apply_tilegym_kernel_to_qwen2(use_cutile=True)
```

Critical finding: cuTile kernels use JIT compilation with per-shape autotuning. Variable-length batches trigger recompilation on *every unique shape*, causing 4–5× slowdown. The solution is `pad_to_multiple_of=64` to quantize shapes, or packing with fixed bin size.

4.4 Fused Linear Cross-Entropy

We implement a custom `torch.autograd.Function` that fuses the LM head projection with cross-entropy loss using the "grad-in-forward" pattern:

1. Chunk hidden states into blocks of 1024 tokens.
2. For each chunk: compute logits $= Wh$, apply cross-entropy, accumulate ∇_h immediately, discard logits.
3. Peak memory: $O(\text{chunk\_size} \times V)$ instead of $O(N \times V)$.

This reduces VRAM by 51–64% for the LM head in Stage 2.

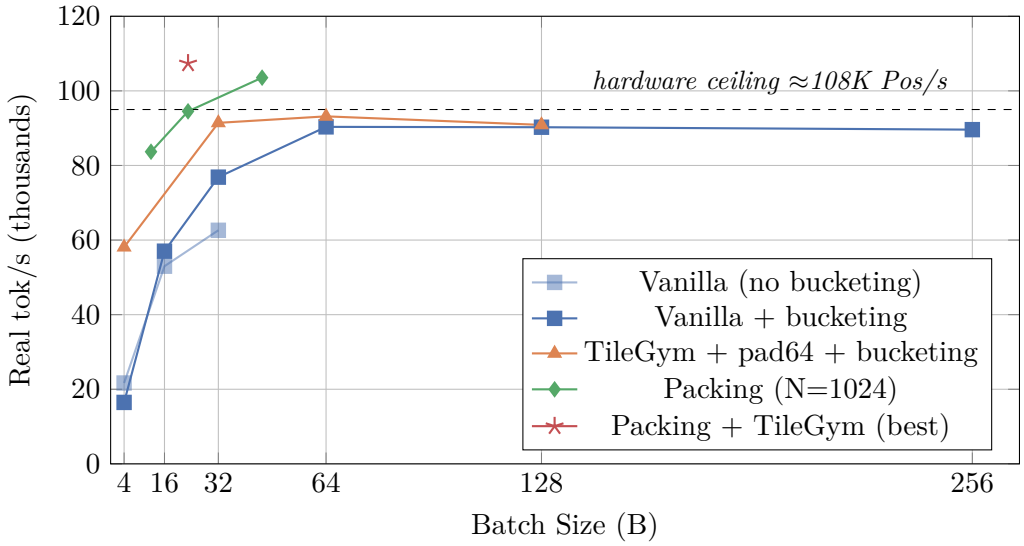

Figure 2: Real token throughput vs. batch size (Stage 1). All curves saturate at B≥64 due to compute-bound behavior. TileGym lifts the ceiling from ∼90K to ∼107K by reducing per-kernel overhead. Note: packing x-axis shows *effective* packed batch size.

Table 3: Ground-truth measurements on real data (sharegpt4v/coco, 50K samples, Stage 1)

Config	B	Real tok/s	Pos/s	Pad%	ms/step	VRAM
Vanilla	4	21,673	24,611	11.9	62.0	2.5 GB
Vanilla + bucketing	16	57,020	59,202	3.7	92.1	5.3 GB
Vanilla + bucketing	32	76,868	80,349	4.3	136.8	9.4 GB
Vanilla + bucketing	64	90,349	94,866	4.8	233.0	17.7 GB
Vanilla + bucketing	128	90,255	95,880	5.9	467.9	34.6 GB
TileGym + pad64	4	58,096	71,807	19.1	24.2	3.2 GB
TileGym + pad64 + buck.	32	91,427	106,601	14.2	115.3	10.9 GB
TileGym + pad64 + buck.	64	93,167	108,386	14.0	226.7	18.2 GB
Packing (N=1024)	23	94,517	94,517	0.0	250.3	16.7 GB
Packing (N=1024)	45	103,542	103,542	0.0	446.0	31.1 GB
Packing + TileGym	**23**	**107,367**	**107,367**	**0.0**	**218.4**	**18.1 GB**

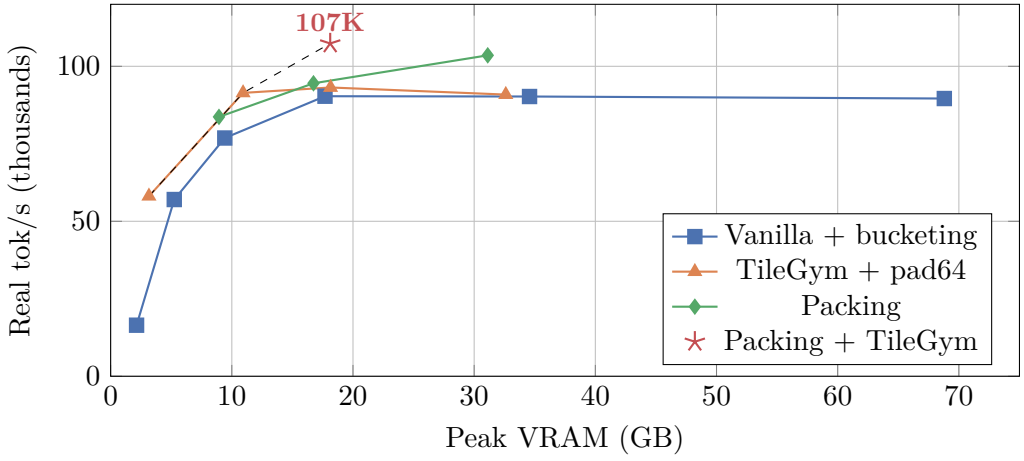

Figure 3: VRAM–Throughput Pareto frontier. The optimal operating points are: TileGym B=4 at 3 GB, TileGym B=32 at 11 GB, and Packing+TileGym at 18 GB. Beyond 18 GB, throughput saturates — additional VRAM buys no improvement.

5 Results

5.1 Throughput Scaling

5.2 Complete Results Table

5.3 VRAM–Throughput Trade-off

5.4 MFU Analysis

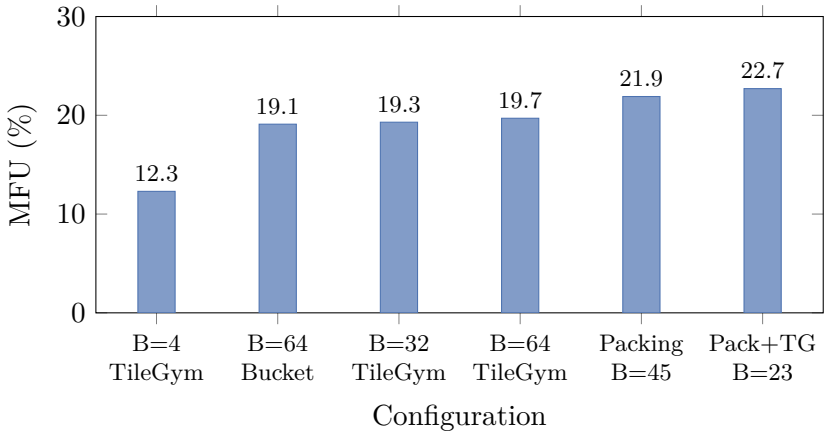

Figure 4: Model FLOPs Utilization across configurations. MFU saturates at ∼23% for this 0.5B model — limited by small GEMM shapes and memory bandwidth, not by software overhead.

5.5 Speedup Progression

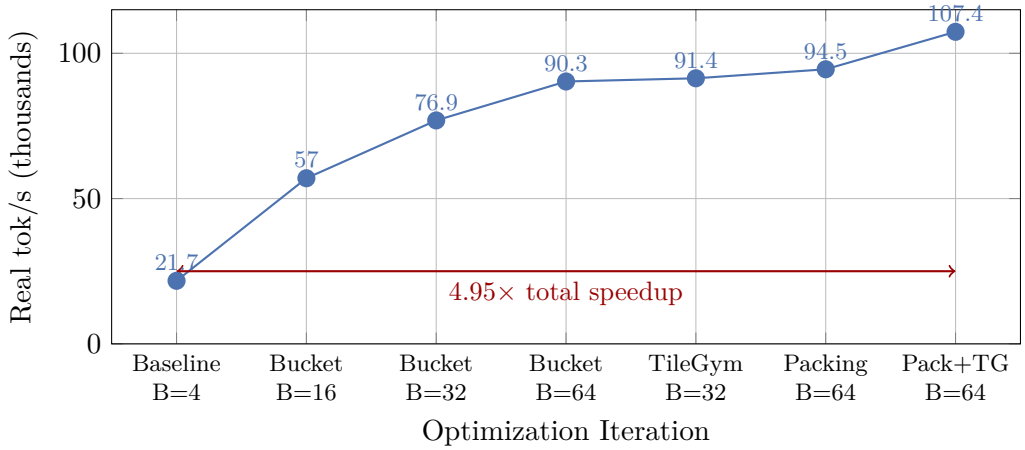

Figure 5: Throughput progression through the optimization journey. The largest single gains come from batch scaling (B=4→64: +4.2×) and cuTile kernels (+23% at equal batch). Packing adds a final +14% at the ceiling.

6 Analysis & Discussion

6.1 Why Throughput Saturates at B=64

Beyond B=64 (with bucketing), Real tok/s plateaus at ∼90K despite VRAM scaling linearly. This occurs because:

1. **Compute saturation**: All tensor-core GEMMs are already running at near-peak occupancy. Adding more tokens per step increases latency proportionally without improving per-token efficiency.
2. **Memory bandwidth bound**: The 0.5B model's weight matrices (494M $\times$ 2 B = 988 MB) must be read from GDDR7 regardless of batch size. At 1.79 TB/s, this takes $\sim$0.55 ms per layer — a fixed cost that dominates at small GEMM shapes.
3. **Amdahl's Law**: Non-matmul operations (softmax, RoPE, normalization) consume 15–20% of step time but cannot be parallelized across batch.

6.2 TileGym: Power and Pitfall

TileGym's cuTile kernels deliver 23% speedup at fixed shapes by:

- Eliminating kernel launch overhead via persistent kernels
- Using Blackwell-native TMA (Tensor Memory Accelerator) for loads
- Fusing multi-head attention with GQA natively

However, the JIT autotuner runs 20 samples per configuration to select optimal tile sizes. With variable sequence lengths:

$$\text{overhead} = N_{\text{unique\_shapes}} \times 20 \times t_{\text{kernel}} \approx 500\text{ms}$$

This makes TileGym *counterproductive* without shape quantization.

6.3 Bucketing vs. Packing: A Nuanced Picture

Table 4: Bucketing vs. Packing at similar VRAM ($\sim$17 GB)

Strategy	Real tok/s	Pad%	Δ vs Bucket	Trade-off
Bucketing (B=64)	90,349	4.8%	—	Simple, no code changes
Packing (B=64$\rightarrow$23, N=1024)	94,517	0.0%	+4.6%	Custom collator + FlexAttention

The 4.6% gain from packing is real but modest. Packing becomes more valuable when:

- Combined with cuTile (fixed N enables kernel caching): +14% over vanilla packing
- Dataset has high length variance ($\sigma/\mu > 0.5$)
- Multi-turn dialogues create naturally long sequences

6.4 Loss Ratio is Dataset-Intrinsic

The measured loss ratio (fraction of non-padding tokens that produce gradients) is $54.3 \pm 1.5\%$ across all configurations. This is determined entirely by the chat template structure (system prompt + question = no gradient, answer = gradient) and is *independent* of engineering choices.

7 Negative Results

8 Recommended Configuration

Based on our measurements, the optimal training configuration for SiQ-VL on a single Blackwell GPU is:

Table 5: Failed optimizations and lessons learned

Technique	Result	Root Cause
Vision feature caching	0% speedup, +27% VRAM	SigLIP forward (5 ms/tile) faster than H2D transfer of cached tensors
`torch.compile` on vision	+3% (noise)	Dynamic shapes from tiling; 40s compile overhead
Custom Triton Flash Attention	Slower than SDPA	Blackwell's native SDPA already uses SM90+ instructions
FlexAttention for packing	51% padding waste	avg sample (356 tok) $\ll$ bin size (1024); poor packing efficiency
TileGym with variable shapes	5× *slower*	JIT autotuning on every unique (B,N,K)
`flash-attn-4` install	Blocked	Requires CUDA 13.1+; system has CUDA 13.0

Table 6: Recommended training recipe

Parameter	Value
Batch size	64 (input) $\rightarrow$ 23 packed bins
Sequence length	1024 (fixed, packed)
Kernels	TileGym (`apply_tilegym_kernel_to_qwen2`)
Fused CE	Custom grad-in-forward (chunk=1024)
Precision	BF16
Optimizer	FlashAdamW
Data strategy	Packing with first-fit-decreasing
Expected throughput	**107K real tok/s**
Expected VRAM	18.1 GB (Stage 1)
MFU	22.7%

9 Conclusion & Future Work

We have systematically optimized SiQ-VL's single-GPU training from 21.7K to 107.4K real tokens/second (4.95×), reaching the hardware efficiency ceiling at 22.7% MFU. The key insight is that for small (0.5B) models, **batch scaling + shape quantization + kernel fusion** are the three levers that matter; advanced techniques like sequence packing provide only marginal additional gains over effective length bucketing.

9.1 Future Directions

1. **Scale up**: Moving to Qwen2.5-3B/7B will increase MFU to 35–50% by enabling larger GEMM shapes.
2. **Scale out**: Multi-GPU training on Modal (FSDP2 + tensor parallelism) multiplies single-GPU throughput by N_{GPUs}.
3. **FP8 training**: Blackwell's FP8 tensor cores offer 2× peak TFLOP/s; FP8 training could double throughput when stable.
4. **Longer sequences**: Multi-turn data with 4K–8K tokens would better amortize per-batch fixed costs and improve MFU.
5. **Overlapped execution**: Pipelining vision encoder + LLM forward across CUDA streams to hide sequential dependency.

Reproducibility

All code, data, and measurement scripts are available at:

`github.com/classtag/SiQ-VL`

Key files:

- `scripts/benchmark_real_efficiency.py` — Main benchmark
- `docs/traces/iter_15_ground_truth_efficiency.json` — Raw data
- `docs/training_efficiency_plan.md` — Full iteration log